Real-Time Fraud Detection in Financial Transactions Using Azure Machine Learning

Dinithi Jayakody

Content

1. Problem Statement and Objective

Financial institutions process millions of transactions every day across credit cards and online banking. Among those transactions, a very small percentage is fraudulent. Despite their low frequency, fraudulent transactions result in significant financial losses. However, detecting fraudulent transactions has several key challenges:

- Detecting fraudulent transactions in real time
- Minimizing false alarms that block genuine customers, and
- Handling highly imbalanced datasets.

To address these issues, this project leverages Azure Machine Learning, which simplifies working with cloud-stored data, provides managed compute resources for model training, and enables serving predictions via a scalable REST API.

The objective of this project is to build a transaction-level fraud detection system using Azure Machine Learning (Azure ML) that:

- Trains a supervised ML model on historical transaction data
- Handles highly imbalanced classes
- Deploys the model as a real-time REST API

2. Dataset

The dataset used in this project is the publicly available Credit Card Fraud Detection dataset from Kaggle, containing anonymized credit card transactions from European cardholders in September 2013. It covers transactions that occurred over a two-day period, comprising a total of 284,807 records. (Dataset)

To protect customer confidentiality, the original features were transformed using Principal Component Analysis (PCA). As a result, the dataset includes:

- 28 principal components (V1–V28)
- A Time feature representing seconds elapsed between each transaction and the first transaction in the dataset
- An Amount feature indicating the transaction value
- A target variable Class, where:
    - Class = 1 indicates a fraudulent transaction
    - Class = 0 indicates a legitimate transaction

The dataset is highly imbalanced, with fraudulent transactions representing only a very small fraction of the total observations.

3. Exploratory Data Analysis

3.1 Missing Values and Data Quality

The dataset was examined for missing values and data inconsistencies.
Findings:

- No missing values
- Data types were consistent across features

3.2 Class Distribution and Imbalance

The dataset is highly imbalanced, with fraudulent transactions representing only a very small fraction of the total observations:

- The majority of transactions belong to Class = 0 (non-fraud) ~99.82%
- Only a very small fraction correspond to Class =1 (fraud) ~0.17%

Figure 1: Class Distribution

Due to this class imbalance, a model could predict all transactions as non-fraud and still achieve high accuracy while completely failing to detect fraudulent activity. Therefore, evaluation metrics beyond accuracy, such as Recall, F1-score, and ROC-AUC, were considered more appropriate for model evaluation.

3.3 Transaction Amount Distribution

Key observations:

- Non-fraud transactions exhibit a wide range of transaction amounts.
- Fraudulent transactions tend to occur at relatively lower transaction amounts compared to legitimate transactions.
- The amount distribution is highly skewed, with many small transactions and fewer large ones.

This insight suggests that transaction amount may contribute useful predictive information, although it should not be used in isolation.

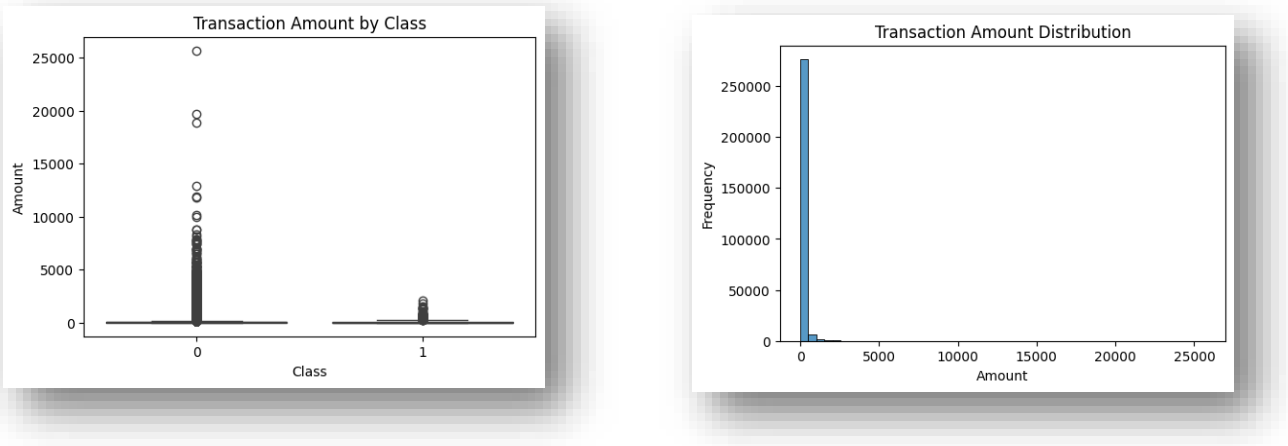

Figure 2: Transaction Amount Distribution

The EDA showed that no missing value imputation was required, as the dataset contained no null values. The Amount feature was scaled to maintain consistency. Outliers were intentionally retained, as extreme values may represent important fraud patterns and could contribute significantly to the model's ability to detect fraudulent transactions.

4. Methodology

4.1 Data Preprocessing

Stratified sampling was used to split the dataset into training and testing sets to preserve the original class distribution. This was essential due to the severe class imbalance, ensuring that both subsets contained representative proportions of fraudulent and non-fraudulent transactions.

4.2 Model Development

Model Selection

Since fraud detection is a binary classification task, logistic regression was selected as the baseline model.

Handling Class Imbalance

As described in data preprocessing section, the dataset was split into training and testing sets using stratified sampling. To further address the severe class imbalance, the logistic regression model was configured with class_weight ="balanced". This automatically adjusts class weights inversely proportional to class frequencies. Without such adjustments, a model could predict all transactions as non-fraud and still achieve high accuracy while failing to detect fraudulent cases. Therefore, performance was evaluated using metrics beyond accuracy to properly assess fraud detection capability.

Training Procedure

The logistic regression model was trained using the PCA-transformed features along with the scaled Amount feature. The model learned the relationship between these features and the target variable (Class). After training, the model was evaluated on the unseen test set to assess its performance.

Model Saving and Reproducibility

Once training was completed, the trained model was saved as a .joblib file. This allowed the model to be:

- Registered in Azure ML model registry
- Reused for deployment
- Loaded during real-time inference via the scoring script

4.3 Model Evaluation

To assess model performance, precision, Recall, F1-score and ROC-AUC metrics were used. Due to the severe class imbalance, accuracy alone was not considered sufficient for evaluating model.

The model achieved 0.92 recall for fraudulent transactions, this means that ~92% of actual fraud cases were correctly identified.

From the confusion matrix: TP – 90, FN – 8. According to that, model is highly effective at detecting fraudulent transactions.

The precision for the fraud class is relatively low (0.06). This means that model flags a significant number of legitimate transactions as suspicious. The model has achieved a ROC-AUC score of 0.97. ROC-AUC measures the model's ability to distinguish between fraud and non-fraud across different classification thresholds. A value close to 1.0 suggests that the model effectively ranks fraudulent transactions higher than legitimate ones.

```
Model Evaluation Metrics:
precision: 0.06056527590847914
recall: 0.9183673469387755
f1_score: 0.11363636363636363
roc_auc: 0.9714551654933217

Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.98      0.99     56864
           1       0.06      0.92      0.11        98

    accuracy                           0.98     56962
   macro avg       0.53      0.95      0.55     56962
weighted avg       1.00      0.98      0.99     56962


Confusion Matrix:
[[55468  1396]
 [    8    90]]
```

Figure 3: Model Evaluation metrics

The evaluation results show that:

- The model successfully detects the majority of fraudulent transactions (high recall).
- The model has strong overall discriminative ability (high ROC-AUC).
- Precision is relatively low due to the severe class imbalance and the model's tendency to prioritize fraud detection.

Overall, the model demonstrates strong potential for fraud detection, though further threshold tuning or advanced algorithms could improve precision while maintaining high recall.

5. Azure ML Pipeline Architecture

To ensure reproducibility and automation, the entire machine learning workflow was implemented using an Azure ML pipeline. The pipeline orchestrates data preprocessing, model training, and evaluation as sequential, dependent steps within a managed cloud environment.

Pipeline Structure

The pipeline consists of three modular components that correspond to the logical stages described in the methodology section:

- Preprocessing Component – Encapsulates data preparation logic and produces processed training and testing datasets.
- Training Component – Consumes the processed data to train the classification model and outputs a serialized model artifact.
- Evaluation Component – Uses the trained model and test data to compute evaluation metrics and generate performance outputs.

Each component is executed as an independent step, with clearly defined inputs and outputs. This ensures that data artifacts flow sequentially through the workflow.

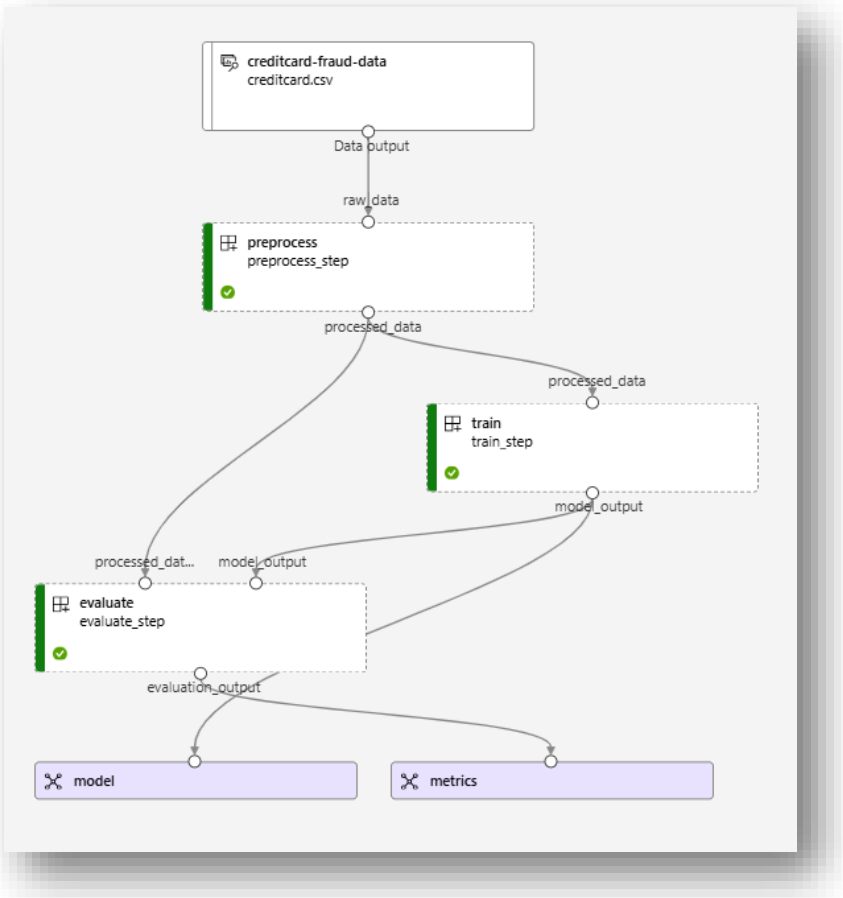

Figure 4: Fraud detection pipeline

Dependency Management and Orchestration

Azure ML automatically manages execution dependencies between steps. The training step is triggered only after successful completion of preprocessing, and the evaluation step depends on both the trained model and processed test data.

Compute and Environment Management

The pipeline was executed on a managed Azure ML compute cluster. The execution environment was defined using a custom Azure ML environment containing the required Python libraries.

6. Model Registration

After successful training and evaluation, the trained model artifact was registered in the Azure ML model registry. Model registration enabled version control, traceability, and deployment readiness within the Azure ML ecosystem.

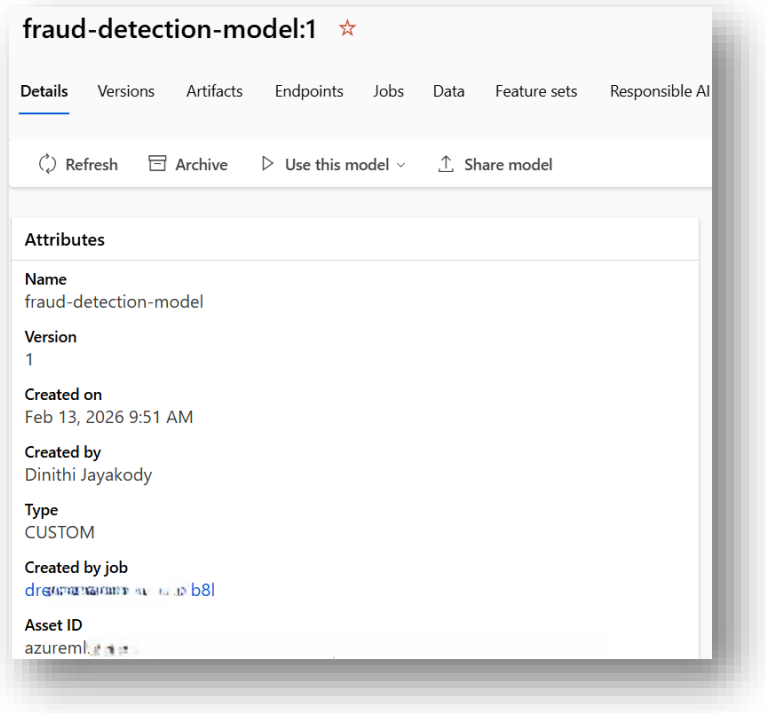

Figure 5: Model Registration

7. Model Deployment

After registering the trained model, it was deployed as a managed online endpoint using Azure ML. This enables real-time fraud detection through a REST API interface.

Deployment Architecture

The deployment process involved:

- Creating a Managed Online Endpoint
- Deploying the registered model as a containerized service
- Defining a scoring script for handling inference requests
- Configuring compute resources for real-time serving

Scoring Script

A custom scoring script was implemented to handle inference requests.

Endpoint Configuration

The endpoint was configured with:

- A managed compute instance
- Traffic routing to the active deployment
- Authentication using secure endpoint keys

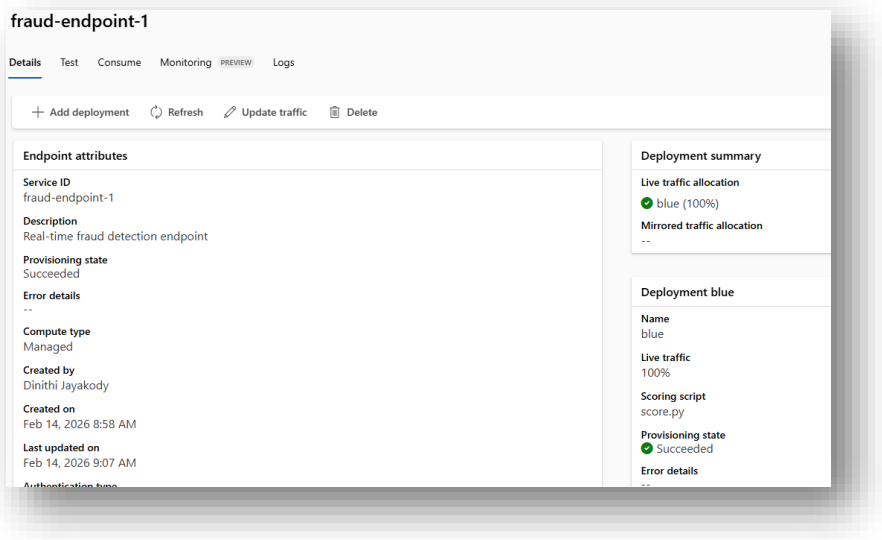

Figure 6: Model Deployment

Real-Time Inference Testing

The deployed endpoint was tested by sending sample transaction data in JSON format. The endpoint successfully returned:

- Predicted transaction class (Fraud or Non-Fraud)
- Probability score representing fraud likelihood

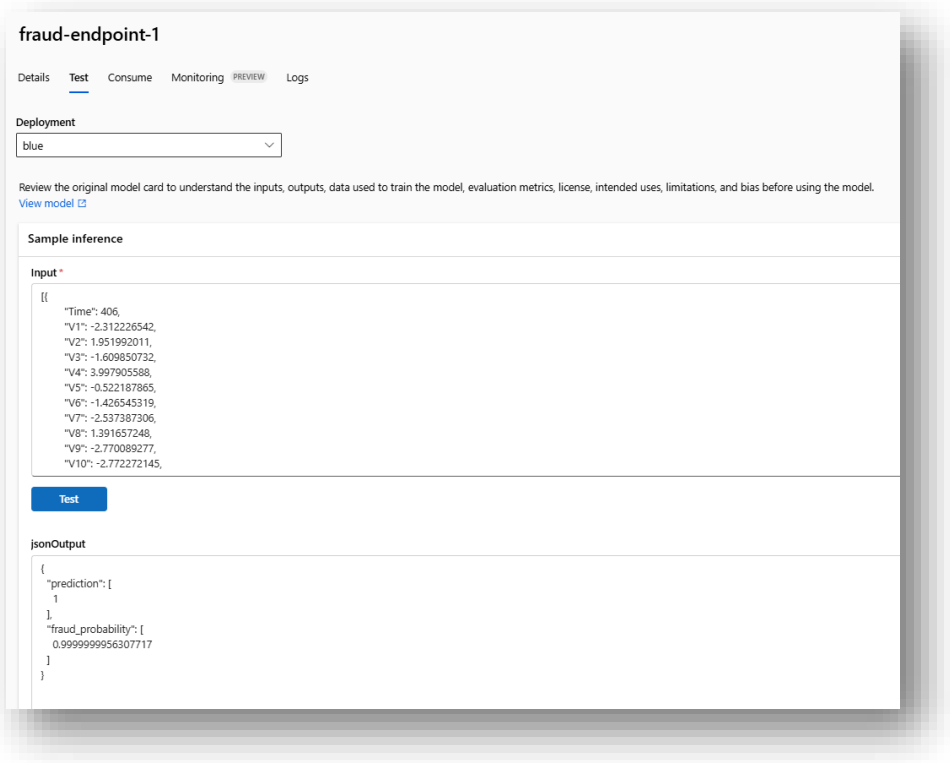

Figure 7: Real Time Inference Testing

Production Considerations

Deploying the model as a managed online endpoint provides:

- Scalability through adjustable compute resources
- Secure access via authentication keys
- Monitoring of request metrics and logs
- Integration capability with external systems via REST API

This deployment approach aligns with real-world financial fraud detection systems, where low-latency and high availability are critical.

8. Conclusion

This project aimed to develop a complete fraud detection system using Azure machine learning. It began with exploratory data analysis, during which severe class imbalance was identified and addressed through appropriate preprocessing techniques.

A Logistic Regression model was then developed and evaluated using metrics suitable for imbalanced classification problems. The model successfully detected the majority of fraudulent transactions. Although it demonstrated strong class discrimination capability, further improvements could be achieved through more advanced algorithms (e.g. XGBoost) and threshold tuning to better balance fraud detection accuracy with customer experience.

Beyond model development, the project automated the entire workflow using Azure ML Pipelines. The trained model was versioned in the Azure ML model registry and deployed as a managed online endpoint, enabling real-time predictions via a REST API.

Overall, this project addressed a real-world financial challenge while demonstrating the complete machine learning lifecycle, progressing from data analysis through to cloud-based production deployment.